

Introduction to PowerShell

Version 5.6b

info@mcg.be



Course Overview

Module 1 : Overview	p.3
Module 2 : Fundamentals	p.13
Module 3 : PowerShell Script Design	p.54
Module 4 : PowerShell Provider	p.91
Module 5 : Manage Windows	p.103
Module 6 : Active directory	p.117
Module 7 : Windows Management Instrumentation	p.127
Module 8 : Powershell and Exchange	p.135
Module 9 : Vmware Powercli	p.141
Module 10 : COM and GUI in PowerShell	p.148
References	p.159

Module 1 : Overview



Content

1. Definition	p.5
2. Features	p.9
3. Installation	p.11

1. Definition

CMD.exe and other traditional shells are text-based, meaning that the commands in these shells take text as input and produce text as output. So manipulate information inside the result is difficult.

Features of PowerShell:

- New command and scripting language
- Provides a command line environment for system administration on windows platform
- Built on top of .NET and so is an object-based shell
- Output produce by a PowerShell command is always an object that is more easy to manipulate
- Installed in Windows\system32

- **What's an Object ?**

An object is a representation of something. If we want to describe an object we must define :

- Properties: describe what an object is
- Methods: what you can do with this object

Everything that describe an object are stored in a class

```
"toto"| get-member #List property and method  
"toto".length # Allow to access a property  
"toto".toupper() # allow to use a method
```

The PowerShell is C# based. So you can reuse some commands existing in C# language.

```
PS >[DateTime]::IsLeapYear(2008)  
True
```



- **What PowerShell can run ?**

There are three types of commands:

- CmdLets: function or command inside PowerShell that complete many of the standard automation tasks
- Script Commands: PowerShell code that lives in a file on disk with a .ps1 extension.
- DOS Commands: Executable file defined by OS

2. Features

PowerShell includes the following key features:

- Include more than 2400 cmdlets
- Colored syntax inside CLI console since PowerShell 5.0
- Allow Tab completion on commands, variables and parameters names
- Case insensitive in command, parameter or variable name
- Use a pipeline character to pass output directly to another command
- Use a security model to prevent execution of unwanted scripts
- Script debugging
- Easy navigation through registry and certificate stores as if we are on a file system
- Extensible interface with module
- Integrated Scripting Environment

- Remote access executable command
- Programming by Modules
- Scheduled and Background jobs
- Event management
- Script internationalization
- Update existing help
- Remove the need to load a module explicitly before calling cmdlets include in it
- PowerShell web access
- Command delegation
- Resilient sessions → Sessions can survive to network failure
- Desired State configuration
-

3. Installation

- **Requirements**

- OS:
 - ✓ For PowerShell 5.1: Windows 7 SP or later, Windows Server 2008 R2 or later
 - ✓ CPU : X86 or X64 bits architecture
- Additional software :
 - ✓ .Net Framework 4.5.2 (For 2008R2 or windows 7)
 - ✓ WinRM 5.1 for PowerShell (For 2012 and later, for 8 or later)
- Determine the version of the PowerShell installed
 - ✓ \$PSVersionTable
 - ✓ HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\PowerShell\3\PowerShellEngine

LAB 1 : Verify version

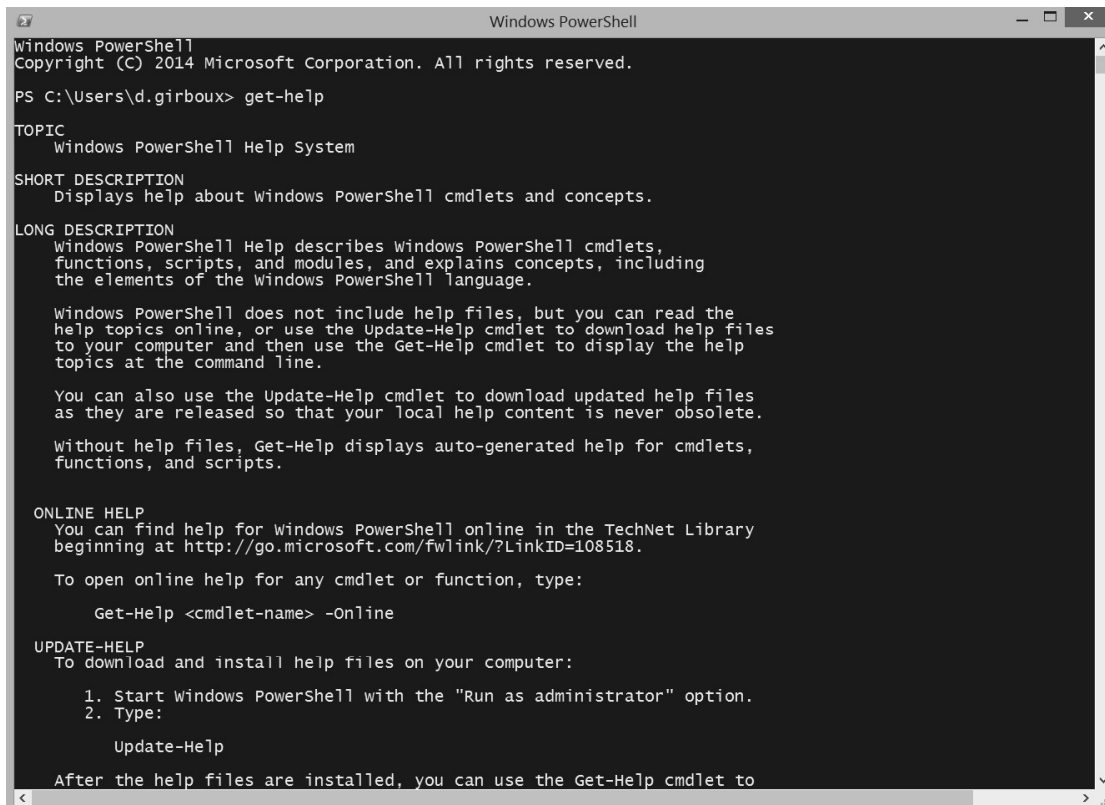
Module 2 : Fundamentals

Content

1. Consoles	p.15
2. Cmdlet	p.21
3. Variable	p.31
4. Basic commands	p.39
5. Other useful commands	p.50

1. Console

- CLI console



```
Windows PowerShell
Copyright (C) 2014 Microsoft Corporation. All rights reserved.

PS C:\Users\d.girboux> get-help

TOPIC
    Windows PowerShell Help System

SHORT DESCRIPTION
    Displays help about Windows PowerShell cmdlets and concepts.

LONG DESCRIPTION
    Windows PowerShell Help describes Windows PowerShell cmdlets,
    functions, scripts, and modules, and explains concepts, including
    the elements of the Windows PowerShell language.

    Windows PowerShell does not include help files, but you can read the
    help topics online, or use the Update-Help cmdlet to download help files
    to your computer and then use the Get-Help cmdlet to display the help
    topics at the command line.

    You can also use the Update-Help cmdlet to download updated help files
    as they are released so that your local help content is never obsolete.

    Without help files, Get-Help displays auto-generated help for cmdlets,
    functions, and scripts.

ONLINE HELP
    You can find help for Windows PowerShell online in the TechNet Library
    beginning at http://go.microsoft.com/fwlink/?LinkID=108518.

    To open online help for any cmdlet or function, type:

        Get-Help <cmdlet-name> -Online

UPDATE-HELP
    To download and install help files on your computer:

        1. Start Windows PowerShell with the "Run as administrator" option.
        2. Type:

            Update-Help

    After the help files are installed, you can use the Get-Help cmdlet to
```



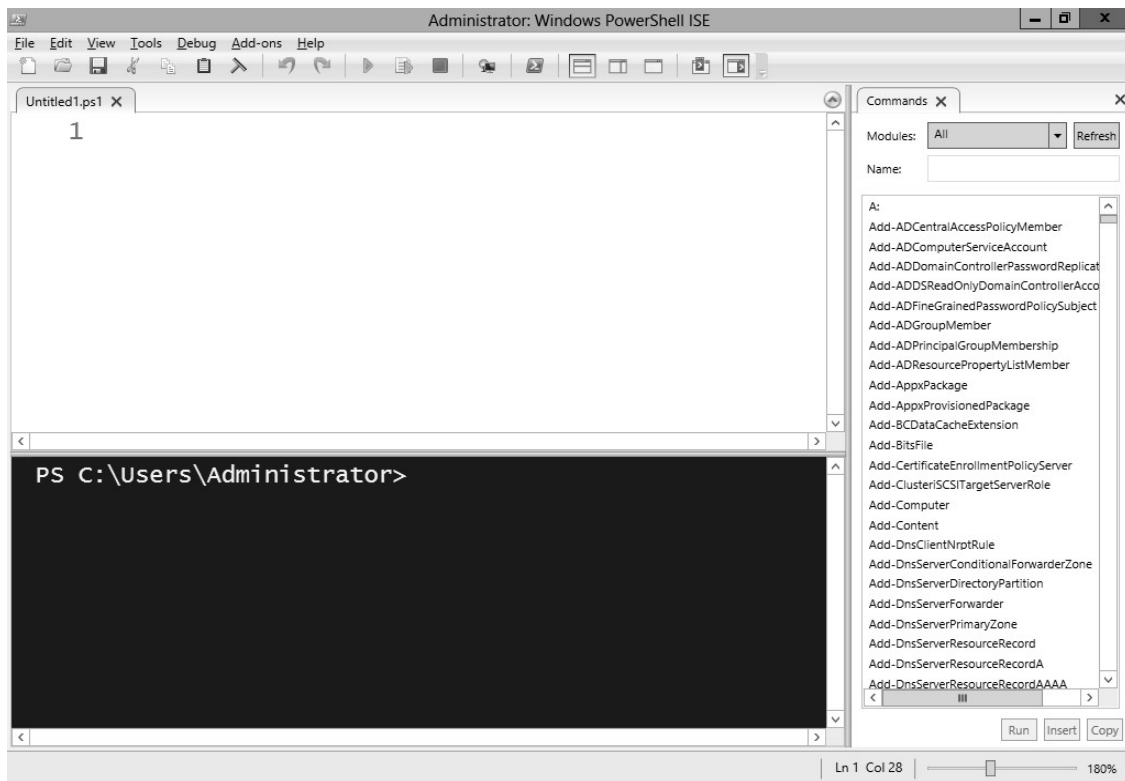
Shortcut	Description
get-history	Display the command history
UP Arrow up or Down Arrow	Scroll up and down your command history
CTRL + Arrow left or CTRL + Arrow right	Move one “word” to the left or right
CTRL + C	Cancel command execution
CTRL + End or CTRL + Home	Deletes all characters from current position to end or up to beginning of command line

- **Integrated Scripting environment**

Graphical development environment and Command Prompt alternative to create scripts or execute command.

Features of Windows PowerShell ISE:

- Debug script with breakpoint
- Multiline editing
- Tab completion
- Syntax coloring
- Selective execution
- Intel sense
- Execute PowerShell on remote computer
- Not in active feature development
- Support only PowerShell 51 and older
- Can use PSEdit command to open script from the CLI in ISE



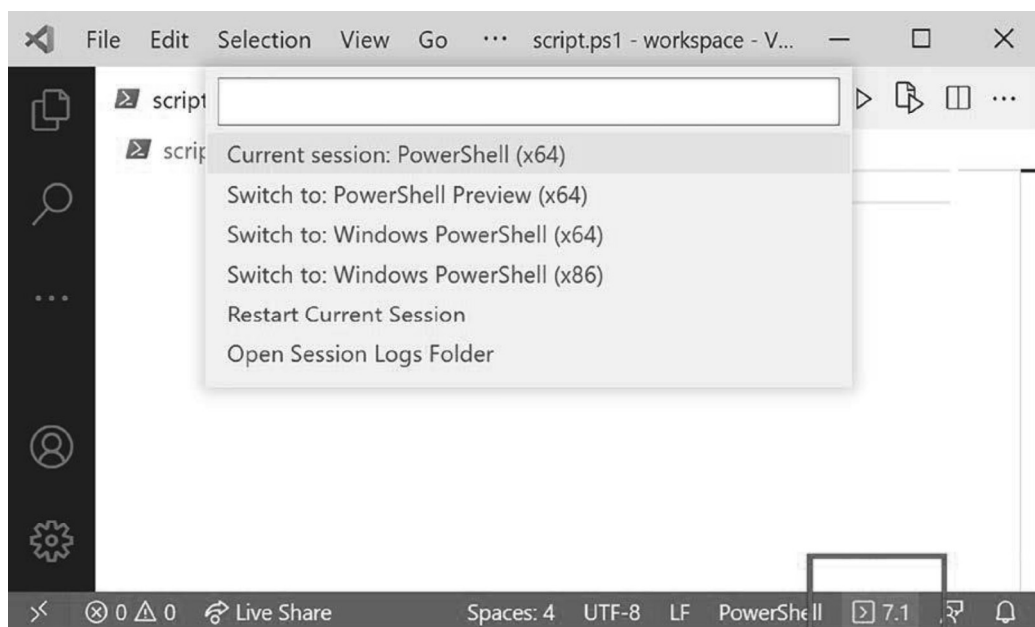


- **Visual Studio code**

Cross-platform script editor by Microsoft to create scripts or execute command.

Features of Microsoft VS code:

- Supports the following PowerShell versions:
 - ✓ PowerShell 7.0 and higher (Windows, macOS, and Linux)
 - ✓ Windows PowerShell 5.1 (Windows-only)
- Recommended by Microsoft
- syntax coloring
- Selective execution by using F8
- Intelisense for cmdlets and more
- Find references of cmdlets and variables
- Can be enhanced by using extensions from the marketplace



2. Cmdlet

In addition to supporting traditional Windows executables, PowerShell introduces a powerful new type of command called a commandlet (or cmdlet).

A commandlet usually consists of three parts:

- A verb
- A noun
- Zero or more parameter

The syntax is like the following :

Verb-noun [-parameter list]



- **Verb part**

The action that describes the task you want to apply to an object. These actions are limited by default but you can create more.

Ex : get, set, add, format, export, new, read, write, ...

- **Noun part**

Identified the system resources and so the object type on which you want to apply the verb.

Ex : Alias, Children, Command, Help, Drive, location, ...

- **Parameters list**

The arguments of these parameters specify the input that the cmdlet accepts, how the cmdlet should perform its actions, and the data that the cmdlet returns to the pipeline. There are three types of parameters :

- **Mandatory or optional Named parameters** : must use the parameter name → order of named parameter is not important
Ex : `Get-ChildItem -path c:\windows`
- **Mandatory or optional Positional parameters** : The position value tells the cmdlet which argument belongs to which parameter. → order of positional parameters is important
Ex : `Get-process system` instead of `get-process -name system`
- **Switch parameters** : does not require an argument. If the switch is present, the value is equivalent to true, while if the switch parameter is absent, it is equivalent to false
Ex: `Clear-RecycleBin -Confirm:$false`



- **Getting Help**

PowerShell includes a built-in help system accessible using the Get-Help command.

Updatable help :

- Allows users to obtain the most recent versions of their help documentation outside of PowerShell on a web service
- Is automatically launch the first time we launch get-help command

```
Get-help <Commandname> | more
Get-help <Commandname> -showinwindows
Get-help about_*
update-help
```




• Splatting

Often used to solve three potential problems in a script:

- Long lines caused by commands that need many parameters
- Conditional use of parameters
- Repetition of parameters across several commands

Individual parameters are written in a hashtable (@{}), and then the @ symbol is used to tell PowerShell that the content of the hashtable should be read as parameters

```
$params = @{  
    Name = 'Internal'  
    SourceLocation = '\\server\share\directory'  
    InstallationPolicy = 'Trusted'  
}  
Register-PSRepository @params
```



- **Alias**

Definition:

- Alternate name for a command
- A command may have more than one alias
- An alias cannot include parameters

Aliases are created for two reasons:

- **Historical:** use older conventional names to facilitate the transition to PowerShell = transitional alias
- **Comfort:** shortcut alternative name for a cmdlet

Get list of alias

```
Get-alias  
Get-command -commandtype alias
```

Create an alias

```
New-Alias -name getting -value Select-String  
Set-Alias -name put -value Write-Host
```



- **Module**

- Collection of cmdlets, functions, variables, and providers
- Extend the functionality of standard PowerShell
- You can save it in format.psm1 and import it in your script
- Since powershell 3.0, an automatic module import occurs whenever a command contained within an installed module is called an initial time in a session
- The module is located inside the folder listed in \$ENV:psmodulepath

Get-Module -listAvailable #list module available but non loaded in the session

Get-installedmodule

Import-Module -name <module name>

Get-Command -module <module name>

Uninstall-module <module name>



- **PowerShell Gallery**

Central repository used to register, search and install modules.

Features :

- Authored by Microsoft and PowerShell community
- Contains
 - ✓ Modules
 - ✓ PowerShell commands
 - ✓ Desired State Configuration (DSC) resources
 - ✓ PowerShell scripts and workflow
- Require
 - ✓ PowerShell 3.0 or later
 - ✓ .NET Framework 4.5 or above

Install-Module -Name PowerShellGet -Force
Save-module -name Carbon -path c:\modules
Find-module -name vmware*

LAB 2 : Commandlet

3. Variable

- **Definition**

Variables is a place to store the result of your commands to manipulate it later. All variable names start with a \$ character.

```
$result = 1+ 5
```

```
#Store in $result 6
```

Remember that in PowerShell, this variable is also an object so you can use the property and the method associated with this object.

```
$a = "Hello world"
```

```
$a.length
```

To display the property of a variable in a write host :

```
$a = "Hello world"  
Write-host "The size of the string is $($a.length)
```

To find out the methods and properties that the variable contains, enter:

```
$result | get-member -membertype property  
$result | get-member -membertype methods
```

By default, PowerShell can determine the type of a variable. But if you want to specify a type you must use casting to handle variable.

```
[int] $result = 3/2  
$result  
2
```


- **Special variable**

- **\$null:** it represents nothing. Assigning \$null to a variable allows you to create that variable but not assign a real value to it .Useful in test
- **Preference variables:** control the default behavior of various output streams in PowerShell
- **Automatic variables :** store state information for PowerShell. Was created and maintained by PowerShell
- **Environment variable :** store information about the operating system environment
- **User variable:** custom variable created by user

`get-variable | select name, description, value | FL *` or

`Dir variable: # List all the variable type except environment`

`Dir env: # List environment variable`



- **Scope**

PowerShell uses scopes to limit access to variables (and other items, such as functions).

Scopes are layered one on top of another; child scopes inherit from parent scopes.

Parent scopes cannot access variables created in child scopes.

There are different level of scope :

- Global Scope : created when you start a new PowerShell session
- Script scope : created whenever PowerShell runs a new script
- Function scope : created whenever you define a function.
- Local scope : current scope and then are relative



- **Regular expression**

Use to perform searches against text.

A regular expression is a pattern consisting of literal characters, an exact match, and special characters that have a variety of different behaviours.

Can be used with `–match` or `–replace` method

Character	Description
.	Any single character
*	The preceding character repeated zero or more times : 'abc' -match 'a*'
+	The preceding character repeated one or more times: 'abc'-match 'a+'
\	Escape character : 'domain\user' -match 'domain\\user'
?	Make the preceding character optional : 'ac' -match 'ab?c'
\t or \n or \r	Tab or Line fee or Carriage return
^	Beginning of a string
\$	End of a string
\b	Word boundary
{exactly}	A fixed number of characters : 'abbbc' -match 'ab{3}c'
{min,max}	A number of characters within a range: 'abbc' -match 'ab{1,3}c'
{min,}	Specifies a minimum number of characters : 'abbbbbc' -match 'ab{2,}c'

Character	Description
[-]	Match a single character to a set of possible characters (ipconfig) -match 'IPv4 Address.+ : *[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+'
' '	or character : 'one', 'two', 'three' -match 'one three'
(){ 3}	Repeating the group of char 3 times : ([0-9]+\.){3}[0-9]+ representing an IP

• Profile

When you create aliases, functions or variables to PowerShell, the lifetime is limited to the PowerShell session.

If you want to keep these changes, you can create a PowerShell profile and add them to the profiles.

The profile is loaded every time that PowerShell starts.

Description	Store profile in
All users	\$PSHome\Microsoft.PowerShell_profile.ps1
Current Users	<My document folder> \WindowsPowerShell\Profile.ps1
ISE all users	\$PSHome\Microsoft.PowerShellISE_profile.ps1
ISE current users	<My document folder> \WindowsPowerShell\Microsoft.PowerShellISE_profile.ps1

4. Basic commands

- **Pipeline**

A way to send output from one command to another command as input.

Series of commands separated by the pipe operator

The goal of this is to simplified writing.

```
Get-service | where {$_.status -eq "Running"}
```

The number of commands in a single pipeline is unlimited.
So be careful of the order of your step.



- **Formatting**

When you run a cmdlet in PowerShell, the type of data displayed as output is determined by the default format of the cmdlet.

➤ Change display on screen

Name	Alias	Description
Format-Table	FT	Display properties under the form of a table (Horizontal)
Format-List	FL	Display properties under the form of a list (Vertical)
Format-Wide	FW	Display one property in columns

To display only specific properties:

```
Get-service | FT name, status -autosize
```


Adjust the display on format table:

```
Get-service | FT name, displayname,status -autosize -wrap
```

To display all properties :

```
Get-service | FL *
```

To format the column :

```
Get-Process | FT  
@{expression="ProcessName"; width=15; label="Name"},  
@{expression="ID"; width=10; label = "ID"},  
@{expression = "CPU"; width=15; label = "CPU Time"}
```

- Select properties for file:
select-object specified properties to be passed along the pipeline or retrieve a specified number of objects from the beginning or end of an array (use when export to a file)

```
Get-service | select name, status |  
export-csv c:\services.csv
```

- **Filtering**

Allows to reduce the result by filtering based on condition:

- The where-object cmdlet : filter the cmdlet according to a specific properties of an object .
\$ _ refers to each of the objects passed into the pipeline from the previous cmdlet

Get-service | where {\$_.status -eq "running"}

- The group-object cmdlet: to group object by using a property

Get-service | group status

Operator you can use in filter expression

Operator	Description
-eq	Tests whether two values are equal.
-ne	Tests whether two values are not equal.
-gt or -lt	Tests whether a first value is greater or less than a second value.
-ge or -le	Tests whether a first value is greater or less than or equal to a second value.
-like or -notlike	Tests whether two values are equals. Can contain wildcards
-match or notmatch	Tests based on a regular expression pattern
-contains	Tests if table contains one or more times the value

- **Sorting**

The sort-object cmdlet allows you to sort objects produced by an earlier element in a pipeline or a variable.

The default method of sorting is ascending. You must specified descending if you want another order.

```
Get-service | sort -descending name
```

To sort and eliminates duplicate elements :

```
Get-Content "list servers.txt" | sort -unique
```

- **Exporting/importing result to a structured file**

You can export/import data to a file. This file can be :

- A CSV file:

```
Get-service | select name, status | Export-Csv  
serviceslist.csv  
$tab=Import-csv serviceslist.csv
```

- A XML file :

```
Get-service | select name, status | Export-CliXML  
serviceslist.xml
```

- A html file :

```
Get-service | select-object name, status |  
ConvertTo-Html | out-file serviceslist.html
```

With tee-object, You write output to a console and a file or variable.

```
get-process | tee -Variable processes #output to console  
plus create a variable processes
```

- **Join char**

Allow to join multiple chars in a table in one line

```
-join @('a','b','c')
```



- **Conversion**

PowerShell has a variety of commands that can be used to convert strings into objects
Convertto- or convertFrom CSV

```
$proc= get-process powershell | select  
name,id,path | ConvertTo-csv  
$people= "David,0123456789,28" | ConvertFrom-  
Csv -Header Name, Phone, Age
```

Convert-string

```
'Michael Caine', 'Benny Hill', 'Raf Vallone' |  
Convert-String -Example 'Michael Caine=MCaine'
```


tostring

```
(Get-Date).ToString('dd/MM/yyyy HH:mm')
```

Convertto-Json

```
Get-Process powershell |Select-Object Name, Id, Path | ConvertTo-Json
```

5. Other useful commands

- **Web requests**

Invoke-webrequest sends an HTTP/HTTPS request method to an endpoint such as a URI or URL.

```
Invoke-webrequest https://www.mcg.be  
Invoke-WebRequest https://expired.badssl.com/ -SkipCertificateCheck  
Invoke-WebRequest -Uri 'https://download.sysinternals.com/files/Handle.zip' -  
OutFile C:\handle.zip
```

Invoke-RestMethods can execute methods exposed by web services

```
Invoke-RestMethod -Uri https://api.github.com/users/powershell/repos
```



- **Archive management**

```
Compress-Archive -Path C:\Invoices -DestinationPath C:\Archives\Invoices.zip
Compress-Archive -LiteralPath C:\Invoices\File1.txt, C:\Invoices\File2.txt -
DestinationPath C:\Archives\Invoices.zip
Compress-Archive -Path C:\Invoices\* -Update -DestinationPath
C:\Archives\Invoices
```

```
Expand-Archive -LiteralPath C:\Archives\Invoices.Zip -DestinationPath C:\
InvoicesUnzipped
```



• Get random

Get-Random will pick a random string or number that have a specific length

We can also use ascii code to convert a code to char

- ✓ Numbers 33-47 + 58-64 + 91-96 + 123-126 represent special char
- ✓ Numbers 48-57 represent [0..9]
- ✓ Numbers 65-90 represent [A..Z] in upper case
- ✓ Numbers 97-122 represent [a..z] in lower case

```
$rand=(33..126) | Get-Random -Count 15 | % {[char]$_} #Allow to generate 15
chars in multiple line
$pass= -join $rand # allow to group char in one line
```

LAB 3 : Basic commands

Module 3 : PowerShell Script Design



Content

1. Definition	p.56
2. Security	p.58
3. User Interaction	p.61
4. PowerShell Language Construct	p.64
5. PSRemoting	p.75
6. Error Handling	p.79
7. Jobs	p.85

1. Definition

A script is a sequence of PowerShell command that is saved in a file. You can run a script with or without parameters, render it mandatory or not.

The extension for Windows PowerShell scripts is .ps1. You can run a script by specifying its full path, or in some cases by using a relative path.

```
PS> .\script1.ps1
```


Script structure

Comment block
Input parameter
Global parameters
Functions
Execution of the script
End of script

To comment a line add `#` at the beginning of the line or `<# ....#>` to comment multiline.

2. Security

PowerShell includes a security feature (Execution policy) that can restrict the use of script.

Execution policy	Description
Restricted	By default (2012 and before or client) Cannot run any script local or remote
AllSigned	Run only script if the script is signed and you trust the CA
RemoteSigned	By default (since 2012R2) Run locally not signed script but external script must be signed
Unrestricted	Run all scripts

In order to modify the execution policy you can use :

➤ PowerShell:

```
Get-ExecutionPolicy    # display the current execution policy
Set-ExecutionPolicy <execution mode>    # change
                                         execution policy
```

➤ Via GPO :

User (or computer) configuration\ Administrative
template\ Windows Component\ Windows PowerShell\ Turn on
script execution

Turn on Script Execution

Turn on Script Execution Previous Setting Next Setting

☐ Not Configured Comment:
☒ Enabled
☐ Disabled

Supported on: At least Microsoft Windows XP or Windows Server 2003 family

Options:

Execution Policy

Help:

This settings lets you configure the script execution policy, controlling what scripts are allowed to run.

If you enable this setting, the scripts selected in the drop down list will be allowed to run.

The "Allow only signed scripts" setting allows script to execute only if they are signed by a trusted publisher.

The "Allow local scripts and remote signed scripts" setting allows any local scrips to run; scripts that originate from the Internet must be signed by a trusted publisher.

The "Allow all scripts" setting allows all scripts to run.

If you disable this setting, no scripts are allowed to run.

Note: This setting exists under both "Computer Configuration" and "User Configuration" in the group policy editor. The "Computer Configuration" has precedence over "User Configuration."

OK Cancel Apply

3. User Interaction

User can interact with script either by argument either by entering information when asked.

➤ Interaction by argument

Scripts can take parameters by using the special variable `$args` which refers to the arguments that are passed to a script.

PowerShell evaluates these arguments by using spaces to separate each argument from the other.

In fact, `$args` is an array starting at 0.

```
script.ps1
    "Hello $($args[0]), you are $($args[1])!"
```

```
Command prompt
ps> .\script.ps1 Jean 18
```

Another way of working is to create a specific function inside the script and put parameters inside this function.

Script2.ps1

Param (\$Name, \$Age)

“Hello \$Name, you are \$Age years old!”

Command prompt

PS> script2 Jean 18

This allows you to specify a default value, force a type, throw an exception if the parameters is missing for the variable.

➤ Interaction by read-host

The Read-Host cmdlet reads a line of input from the command line. It can take parameters like AsSecureString which can be used to mask what you type.

```
$name = read-host "Enter your name"  
$password = read-host "Enter your password "  
                                     -AsSecureString  
write-host "Your name is $name"  
write-host "Your password is $password"
```

4. PowerShell Language Constructs

- **Array**

An array contains a collection of objects. Each entry in the array is called an element, and each element has an index (position). Indexing in an array starts from 0.

Basic creation : `$array=1,2,3` or `@(1,2,3)`

Range creation : `$array= 1..10`

To set the value of an array element to null, enter:

```
$array[1]=$null
```

To add an element in a array, enter:

```
$array=$array + 10 or  
$array3 = $array1 + $array2
```


- **Hashtable**

Hashtable is an associative array or an indexed array. Values in the Hashtable are added with a unique key. Each key has a value associated with it

```
$hashtable=@{} #creation
$hashtable.key1 #access element
$hashtable.keys # list all the keys
$hashtable.values #list all the values
$hashtable.Add("Key1", "Value1") or
    $hashtable.new='toto' #add element
$hashtable.remove('key1') #remove element
```



• Collection

- Like array but with more features
- Type of collections :
 - ✓ System.Collections.Generic.List
 - ✓ System.Collections.Generic.Dictionary
 - ✓ System.Collections.Generic.Queue
 - ✓ System.Collections.Generic.Stack

```
$list= [System.Collections.Generic.List[String]]::new() #creation  
$list.Add("wks1") #Add element at the end of the list  
$list.Insert(0, "wks2") #Add element at the first position  
$list[1]= " wks3"  
$list.removeat(1) or $list.remove("wks1) #remove element
```

- **Conditional expression**

PowerShell supports two conditional expressions:

- The if-then-else statement

```
$item = get-item 'C:\Windows\System32'  
If ($item.mode -like '*d*')  
    {write-host this is a directory}  
Else  
    {write-host 'this is a file'}
```



You can also use the elseif statement

```
[int]$a = read-host 'Enter a number between 1 and 10'
if ($a -lt 3)
    {write-host $a is less than 3}
Elseif ($a -le 5)
    {write-host $a is between 3 and 5 inclusive}
Else
    {write-host $a is more than 5 exclusive}
```

➤ The switch statement

```
[int]$menu = read-host ' Enter a number between 1
and 4'
switch ($menu)
{
    1 {write-host 'You entered menu 1'}
    2 {write-host 'You entered menu 2'}
    3 {write-host 'You entered menu 3'}
    4 {write-host 'You entered menu 4'}
    Default {write-host 'You didn't enter a number
between 1 and 4'}
}
```

- **Looping constructs**

Use to repeat a sequence of command to be executed multiple times until a specified condition is true or browse a collection.

PowerShell supports three looping constructs:

➤ The for loop

```
for ($i = 0; $i -lt 10; $i++)  
{  
    write-host "The value of `i is $i"  
}
```

➤ The while loop :

```
$i = 0
while ($i -lt 5)
{
    write-host "The value of `"$i" is $i"
    $i++
}
```

➤ The foreach loop:

```
$a = "a", "b", "c", "d"
foreach ($i in $a)
{
    write-host "The value of `"$i" is $i"
}
```

• **Function**

- Use to repeat a sequence of command to be executed multiple times
- Identify by a unique user-assigned name following by bracket () and Demarcated with brace symbols {}
- Are stored automatically within the drive called Function:
- Can be defined in Script, function, pipeline, module
- Support common parameters like Confirm and WhatIf
- Has the ability to take a pipeline input (linked to the \$input variable)
- Define parameters
 - ✓ Allow to pass argument from principal code to function
 - ✓ Separated by a comma
 - ✓ Defined as a block using the param keyword or directly between the bracket
 - ✓ Type can be assigned to a parameter
 - ✓ A default value can be use when a parameter value is not explicitly passed when calling the script

- Divide function into named blocks (optional) to be use in a pipeline :
 - ✓ Begin : Runs before pipe line processing starts
 - ✓ Process : runs once for each value received from the pipeline
 - ✓ End : executes after process has acted on all objects in the input pipeline
 - ✓ Cleanup : block of commands that are execute everytime even if a a terminating error is raised
 - ✓ if none of these blocks are declared, content is in the end block
- Scope :
 - ✓ A function only exists within the scope which it created
 - ✓ A variable cannot be called outside the function scope

```
Function get-file()  
{ param($dir)  
  $listfile= get-childitem $dir  
  return $listfile  
}  
  
$file=get-file 'c:\windows'
```

5. PSRemoting

To execute command on a remote computer. In PowerShell there are two ways to do that :

- Include in command

```
Get-help * -parameter computername
```

- PowerShell remoting

- ✓ Uses a communications protocol called Web Services for Management (WS-MAN).
- ✓ Works over http or https
- ✓ Must be activated on the destination machine via the command

```
Enable-PSremoting
```

- ✓ This command
 - Create an exception in the Windows Firewall for incoming TCP traffic on port 5985
 - Create an HTTP listener on port 5985 for all local IP addresses
 - Set the WinRM service to start automatically and restart it
 - Register up to four default endpoints for use by Windows PowerShell

➤ Once activated we can do

- ✓ Remoting one-to-one

```
Enter-PSSession -computername <computername>  
Exit-PSSession
```

OR

```
Connect-PSSession -computername <computername>  
Disconnect-PSSession
```

✓ Remoting one-to-many

```
Invoke-Command -computerName <computername1>,  
<computername2>,... - scriptblock {commands}
```

➤ Trust the remote access

- ✓ If both machine are in domain → the trust is automatically done → no action
- ✓ If one machine are in domain or workgroup → must use the trustedhost variable

```
$curList = (Get-Item WSMan:\localhost\Client\TrustedHosts).value  
Set-Item WSMan:\localhost\Client\TrustedHosts -Value  
"$curList, Server01"
```

➤ Copy items between sessions

#copy a file to local syetm from a remote session

```
$session1 = New-PSSession PSTest1
```

```
Copy-Item -Path C:\temp\doc.txt -Destination C:\Temp -FromSession  
$session1
```

#copy a file to a remote session

```
$session2 = New-PSSession PSTest2
```

```
Copy-Item -Path C:\temp\doc.txt -Destination C:\Temp -ToSession  
$session2
```

6. Error Handling

- **\$? variable**

Contains the value True if the preceding statement executed successfully otherwise false

Get-Item c:, nosuchfile, # \$? Will be false because there is no suchfile



- **Error behavior**

Specify the action to take in response to an error occurring. This action can be take :

- Globaly thanks to \$ErrorActionPreference
- Selectively by cmdlet thanks to parameter ErrorAction

The following options are available:

Option	Description
SilentlyContinue	Don't display an error message and continue
Continue	By default Display an error but continue
Inquire	Ask the user what to do
Stop	Terminate the action with error

- **Error storage**

To access the error in the SilentlyContinue mode, you have two possibilities :

- Globally thanks to \$Error: \$error[0] is always the last error
- Selectively by cmdlets thanks to parameter ErrorVariable

```
Get-ChildItem -Path c:\temp\nosuchfile #Store error in  
table $error
```

```
Get-ChildItem -Path c:\temp\nosuchfile –  
errorvariable $err #Store error in a table called $err
```

```
$Error[0] | FL * -force #Display all property of the error  
$error.clear() #Clear all table contents
```



- **Try/catch**

When using a Try/Catch/Finally block, the command you wish to execute is placed in the Try block. If an error occurs when the command executes, the error will be written to the `$_error` variable, and script execution moves to the Catch block.

The Finally block sequence always runs, regardless if an error is generated or not

Errors come in two types:

- **Terminating error**

- ✓ An error that will halt a function or operation.
- ✓ Can be caught and handled.

- **Non-terminating errors**

- Allow PowerShell to continue and usually come from cmdlets or other managed situations.
- Cannot be caught by Try-Catch-Finally.

```
Try {$authorizedusers=get-content '\\servername\share\users.txt' –  
erroraction stop  
}  
Catch  
{ $errorMessage=$_.exception.message  
  $faileditem=$_.exception.itemname  
  "Error : " + $errorMessage + " on " + $faileditem  
  
}  
Finally  
{  
  $Time=Get-Date  
  "This script made a read attempt at $Time" | out-file  
    c:\logs\logging.log -append  
}
```



- **Event log**

Using the event logs enables you to capture the messages for future analysis.

The best is to create a custom eventlog

```
New-EventLog -LogName CustomLog -Source Scripts
```

```
#Create the eventlog
```

```
Write-EventLog -Message "Starting. Value is $_" -LogName Customlog  
-Source Scripts -EntryType Information -EventId 1001
```

```
#Create an event in event log
```

```
Get-EventLog -LogName customlog -newest 10 -entrytype information
```

```
#View the 10 last event of type information
```



- **Record of session**

Start-Transcript creates a record of PowerShell Sessions and store it into a txt file

```
Start-Transcript -Path C:\Temp\Log1.txt  
Stop-Transcript
```

7. Jobs

- **Background job**

When you run a command in a PowerShell session, the session is effectively blocked until the command completes and returns its results.

Background jobs allow you to run tasks asynchronously you get the prompt back to continue working while PowerShell runs the job in the background.

➤ Start job

```
Start-Job -ScriptBlock { $events=Get-Eventlog -LogName security  
}  
Start-job -filepath ".\script.ps1"  
  
get-help * -Parameter asjob
```

➤ Check job

```
Get-job | fl  
Get-Job -Name ServiceCheck -IncludeChildJob
```

➤ Get job result

```
Receive-Job -id 13
```

➤ Stop job

```
stop-job -id 13
```

➤ Wait until the end of the job and display the result

```
$jsb = { foreach ($i in 1..10) { Start-Sleep 10; "i is $i" } }  
Start-job -scriptblock $jsb | wait-job | receive-job
```

➤ Remove job in the list

```
Remove-job *
```

- **Remote jobs**

Run a command on Multiple computers at the same time. Each computer is considered to be a child job with his property and result

```
$jobs= {$events=get-Eventlog -LogName security}
$j = Invoke-Command -ComputerName localhost, W16DC01,
W16TGT01 -ScriptBlock {$j} -AsJob

$j | Format-List Name, ChildJobs #display main job and childjobs
Receive-Job -Id 12 -Keep #get the result of a childjob
```


- **Scheduled jobs**

Ability to execute long-running tasks asynchronously

```
$trigger = New-JobTrigger -Daily -At '3 am'
```

```
$options = New-ScheduledJobOption -HideInTaskScheduler -  
RequireNetwork
```

```
Register-ScheduledJob -Name DailyRestart -ScriptBlock { Get-  
Process ; Restart-Computer -force } -Trigger $trigger -  
ScheduledJobOption $options
```

```
Get-Scheduledjob
```

```
Remove-Job -Id 12
```

LAB 4 : Script design

Module 4 : PowerShell providers

Content

1. Definition	p.93
2. Usage Example : The File System	p.95
3. Usage Example : The Registry	p.100

1. Definition

To access, retrieve, and manipulate data from a data store, PowerShell uses providers.

A provider is a .NET program that makes available data from a data store as if they were mounted drives.

PowerShell drive use an identifier followed by a semicolon like C:, D:, Env:, HKCU:

```
PS C:\> get-psprovider
```

Name	Capabilities	Drives
----	-----	-----
Registry	ShouldProcess, Transactions	{HKLM, HKCU}
Alias	ShouldProcess	{Alias}
Environment	ShouldProcess	{Env}
FileSystem	Filter, ShouldProcess, Credentials	{C, D, F, G...}
Function	ShouldProcess	{Function}
Variable	ShouldProcess	{Variable}

Features of a provider :

- Creation and delete of a new root
- Work with items (get-item, clear-item,...)
- Operation on item container (get-childitem, new-item,..)
- Navigation operation (move-item,...)
- Operation on item (add-content, get-content,...)
- Item property operation (get-itemproperty, set-itemproperty,...)



2. Usage Example: The File System

- **Command and alias**

Command	Alias	Definition
Get-psdrive	N/A	List available psdrive
New-psdrive	N/A	Create a psdrive
Set-location	CD	Navigate into file system
Get-location	N/A	Display the current path
Get-childitem	DIR, LS	Explore content of a folder
Test-path	N/A	Test if a file or folder exist
\$pwd	N/A	Print working directory

- **Explore the content of the drive**

```
Get-childitem -path "C:\Program Files\microsoft" -recurse
```

#To explore a specific folder

```
Get-childitem *
```

#To explore the content of the current folder

- **Create item**

```
New-item -path C:\log -type directory
```

```
New-item -path C:\log\logfile.txt -type file
```

```
New-Item temps -ItemType SymbolicLink -Value c:\temp
```




- **Add content to a file**

```
$TextToAdd="Something to add in a file"  
$logfile='c:\logs\logfile.txt'  
New-item -path $logfile -itemtype file  
Add-content -path $logfile -value $TextToAdd
```

- **Get the content of an item**

```
Get-content C:\Lines.txt #See the content of a file  
Get-content C:\Lines.txt -totalcount 5 #See the content of the first 5  
lines  
Get-content C:\Lines.txt | measure-object -line -word -  
character #Get statistics on text file
```

- Find folder and files

```
get-childitem "test" -recurse | where {$_.Mode -match  
"d"} #Find folders  
get-childitem "test" -recurse | where {$_.Mode -  
notmatch "d"} #Find files  
test-path "C:\Program Files\Microsoft"  
#Test the existence of the folder  
[System.IO.File]::Exists('c:\pagefile.sys') #Another way to test  
the existence of a file
```



- **The fileinfo object**

To get info on file or to modify fileobject you can use properties and method of the fileinfo object method :

```
$a=get-childitem "C:\file.txt"  
$a.length  
$a.set_IsReadOnly(0)
```

3. Usage Example : The Registry

Windows PowerShell use the same cmdlets as file system to explore and modify registry

- **Navigation**

Set-location HKLM:

Set-location Software\Microsoft\PowerShell\1\ShellIds:

- **Create a key**

new-item PersonalKey



- **Access or Modify property of a key:**

Get-itemproperty *

Set-itemproperty HKCU:\Software\Microsoft\Notepad –
Name StatusBar –Value 1

LAB 7 : Providers

Module 5 : Managing Windows



Content

1. Basic configuration	p.105
2. Manage network	p.107
3. Manage file and disk	p.111
4. Manage sharing and permission	p.113

1. Basic configuration

- **Rename computer**

```
Rename-Computer -NewName "Server044" -Restart
```

- **Activate remote desktop**

```
Set-ItemProperty -Path  
'HKLM:\System\CurrentControlSet\Control\Terminal Server'  
-name "fDenyTSConnections" -value 0
```

- **Disable firewall**

```
Set-NetFirewallProfile -Profile Domain,Public,Private -  
Enabled False
```

- **Windows server roles and features**

Get-WindowsFeature | Where Installed #Display all roles and features installed

Install-WindowsFeature -Name Web-Server

-IncludeAllSubFeature -IncludeManagementTools

#Install web serv component + all subfeatures

Uninstall-WindowsFeature -Name Web-Server #Remove roles and all subfeatures

2. Managing network

- **Manage physical interface**

```
$adapter=get-netadapter | where {$_.status -eq 'up'} #List
enabled NIC
$adapter | disable-netadapter #Deactivate NIC
$adapter.ifindex #Get the index of the interface
```

- **Retrieve ip configuration**

```
$adapter | Get-NetIPConfiguration #Get basic ip configuration
$adapter | Get-NetIPAddress -AddressFamily ipv4 #Get advanced
ip configuraiton
```



- **IP configuration**

```
New-NetIPAddress -InterfaceIndex 12 -IPAddress
192.168.100.70 -PrefixLength 24 -DefaultGateway
192.168.100.1 #Configure interface with new ip configuration
Set-NetIPInterface -InterfaceIndex 12 -Dhcp enabled # activate
DHCP
```

- **Route configuration**

```
get-netroute -InterfaceIndex 12 -AddressFamily ipv4 #Retrieve
ipv4 route on specific interface
New-NetRoute -DestinationPrefix "10.0.0.0/24" -
InterfaceIndex 12 -NextHop 192.168.0.1 #add route
Remove-NetRoute -InterfaceIndex 12 -DestinationPrefix
10.0.0.0/24 -Confirm #Delet a specific route
```

- **DNS configuration**

```
Get-DnsClientServerAddress -interfaceindex 12 #View DNS ip
Get-DnsClient -InterfaceIndex 12 | fl * # view DNS configuration
except IP
Set-DnsClientServerAddress -InterfaceIndex 12
    -ServerAddresses 8.8.8.8,8.8.8.4 #Configure DNS ip
Set-DnsClientServerAddress -InterfaceIndex 12
    -ResetServerAddresses #Change DNS from local config to
DHCP config
```

- **Tools**

```
Test-Connection 8.8.8.8 #ping
Test-netConnection 8.8.8.8 -TraceRoute # tracert
Test-netConnection 8.8.8.8 -CommonTCPPort smb # ping + test
    If smb port is open
Clear-DnsClientCache #Ipconfig /flushdns
Resolve-DnsName -Name google.fr #Nslookup for A or AAA
    record
(Resolve-DnsName -Name www.google.fr -type A).ipaddress
    #Get the ipaddress only
Resolve-DnsName -Name google.fr -type mx #Resolve request
    for the mx record
Get-NetTCPConnection #Netstat
```

3. Manage File and disk

- **File Catalog**

- Form of File Integrity Monitoring
- Generates and stores SHA1 hashes for each file within a folder structure and writes the result to a catalog file.

```
New-filecatalog -path c:\temp\fc -catalogfilepath  
c:\security\catalog.cat #Create a catalog
```

```
$result=Test-FileCatalog -path c:\temp\fc -  
catalogfilepath c:\security\catalog.cat -detailed $true #Test  
if sha1 is not changed → result is a table
```

```
Get-FileHash C:\ISO\windowsserver.iso -Algorithm  
SHA256 | Format-List #Test the hash of one file
```

- **Manage disk**

Get-disk # Retrieve physical disk information

```
#Get disk space of all disk
```

```
$disks= Get-WmiObject -class win32_logicaldisk
```

```
Foreach ($disk in $disks) {
```

```
$driveletter = $disk.DeviceID
```

```
$sizeGB = [System.Math]::Round(($disk.size / 1GB),2)
```

```
Write-host "$driveletter : $sizeGB"
```

```
}
```


4. Manage Share and permissions

- **Manage share**

Get-SMBShare #Equivalent to net share

New-smbshare -name Virtual -path c:\vm #Create a share
with default permission

- **Access to share**

Get-SMBMapping # Equivalent to net use

New-SmbMapping -LocalPath r: -RemotePath

\\srv1\shared #Create a mapping share

Get-SmbConnection #View share connection

Get-SmbOpenFile #View share openfile

- **Manage share permission**

```
Get-SmbShare shared | Get-SmbShareAccess #Get  
permission on share  
Grant-SmbShareAccess shared -AccessRight Read  
-AccountName user1 #Give permission to user1  
Revoke-SmbShareAccess shared -AccountName  
'Everyone' #Remove permission
```



- **Manage NTFS permission**

Add permission to a file

```
#get ACL
```

```
get-acl c:\temp | select Select-Object -ExpandProperty  
Access | fl *
```

```
#Put new ACL
```

```
$ace= new-object  
System.Security.AccessControl.FileSystemAccessRule  
"user1","FullControl","Allow"  
$acl.SetAccessRule($ace)  
$acl | Set-Acl services.csv
```

```
#Take ownership
```

```
takeown /f services.csv
```

```
#Disable inheritance  
$acl=get-acl 'services.csv'  
$acl.SetAccessRuleProtection($True, $True)  
$acl | Set-Acl services.csv
```

Module 6 : PowerShell and Active Directory



Content

1. Introduction	p.119
2. Search AD Object	p.120
3. Manage AD Object	p.123

1.Introduction

Since Windows 2008R2, PowerShell introduces a new module called activedirectory. This module contains cmdlets in order to manage active directory.

- To use active directory module:

Import-module activedirectory

2. Search AD Object

- **Search user, group, computer, OU**

➤ To search users, group, computer :

```
Get-ADUser -searchbase "dc=Furniture,dc=com" -filter *
```

```
Get-ADGroup -searchbase "dc=Furniture,dc=com" -filter {groupcategory  
-eq "Security" -and GroupScope -eq "Global"}
```

```
Get-ADComputer -filter {operatingSystem -like "Windows 7*"}
```


- To get information about one user :

```
Get-ADUser -identity user1 -property *
```

- To search users, group or computer based on a criteria like account and password status:

```
Search-ADAccount -accountdisabled -usersonly  
Search-ADAccount -accountexpiring -timespan 6.00:00:00  
Search-ADAccount -AccountInactive -TimeSpan 90.00:00:00  
-UsersOnly
```

- **Obtain information about AD objects**

- To obtain functional level:

Get-ADrootDse | ft domainfunctionality, forestfunctionality -a

- To obtain FSMO role:

Get-ADForest furniture.com | FT SchemaMaster, DomainNamingMaster
Get-ADDomain furniture.com | FT PDCEmulator, RIDMaster,
InfrastructureMaster

- To obtain Global Catalog:

Get-ADForest furniture.com | FL GlobalCatalogs

- To obtain list of AD site:

Get-ADObject -LDAPFilter "(objectclass=site)" -SearchBase
"cn=configuration,dc=furniture,dc=com"

3. Manage AD Object

- **Manage OU**

- Create OU:

```
$ou="test"  
New-ADOrganizationalUnit $ou -path "dc=furniture,dc=com"
```

- Modify OU:

```
$ou="Ou=test,DC=furniture,DC=com"  
Set-ADOrganizationalUnit -identity $ou -description "Users for testing"  
-ProtectedFromAccidentalDeletion $false  
OR  
$ou="Ou=test,DC=furniture,DC=com"  
$inst=get-ADOrganizationalUnit $ou  
$inst.country="BE"  
Set-ADOrganizationalUnit -instance $inst
```

- **Manage user**

- Create user:

```
$pass=(convertto-securestring -asplaintext 'test1234=' -force)
New-ADUser -SamAccountName "Toto" -GivenName "Titi"
           -DisplayName "Andy Rose" -Path 'OU=test,DC=furniture,
           DC=com' -accountpassword $pass -userprincipalname
           'toto@furniture.com' -Enabled $true
```

- Modify User:

```
$user="CN=Toto,Ou=test,DC=furniture,DC=com"
Set-ADUser $user -description "Users created via script"
```

- **Manage group**

- Create group:

```
New-ADGroup -Name 'GG1' -SamAccountName 'GG1'  
            -groupCategory Security -GroupScope Global  
            -path 'Ou=test,DC=furniture,DC=com'
```

- Modify group membership :

```
Add-ADGroupMember gg1 user1,user2
```

Or

```
Add-ADPrincipalGroupMembership user1 -memberof GG1
```

LAB 5 : Interaction with Active Directory

Module 7 : Windows Management Instrumentation

Content

1. Introduction	p.129
2. Working with WMI	p.130

1. Introduction

CIM is the way to access and manage some aspect of hardware or software (local and remote)

Windows Management Instrumentation (WMI) is the Microsoft implementation of CIM. So WMI represents the information of a computer in the form of classes.

WMI provides classes for nearly everything:

- Windows Services: win32_Service class
- Windows Process : win32_Process class
- CPU : win32_Processor class
- Date and time : win32_Date class
- ...

2. Working with WMI

- **Obtain information**

Get-WmiObject –list	#list all the wmi32 class
Get-WmiObject Win32_bios	#list info about a particular class

➤ Filter the class:

Get-WmiObject –list where {\$_.name match 'time'} OR Get-WmiObject –query 'select caption, commandline from Win32_Process where name like "p%“'
--



- **Access to information**

- On local computer

```
$memory = (Get-WmiObject  
Win32_ComputerSystem).TotalPhysicalMemory  
$memoryInMB = $memory /1MB
```

- On remote computer

```
$cred = Get-Credential  
$memory =(Get-WmiObject Win32_ComputerSystem  
-computer <machineDistant> -cred  
$cred).TotalPhysicalMemory  
$MemoryInMB = $memory /1MB
```



- **Modify properties**

Most of the properties that you find in WMI objects are read-only. There are few, though, that can be modified.

```
$drive = [wmi]"Win32_logicaldisk='D:'"  
$drive.VolumeName = " Data"  
$drive.Put()
```

- **List method on WMI object**

```
Get-WmiObject Win32_ComputerSystem | Get-Member  
-MemberType method
```



- **Some useful commands for windows**

Get-CimInstance -ClassName Win32_BIOS #access to bios information

Get-CimInstance -ClassName Win32_ComputerSystem #get information about local computer

Get-CimInstance -ClassName Win32_QuickFixEngineering #get list of installed update

Get-CimInstance -ClassName Win32_LogicalDisk -Filter "DriveType=3" #get available disk space

LAB 6 : WMI

Module 8 : Exchange

Content

1. Introduction	p.136
2. Working with database	p.138
3. Working with mailbox	p.139

1. Introduction

Exchange management shell is the extension of powershell that add exchange module.

2. Working with database

➤ Create database

```
New-MailboxDatabase -Name "DB01" -EdbFilePath  
"E:\Databases\DB01\DB01.edb" -LogFolderPath "F:\Logs\DB01" -  
Server Server01
```

➤ Move database

```
Move-DatabasePath -identity <default database> -edbfilepath  
E:\databases\db01\database01.edb -LogFolderPath  
"F:\logs\DB01"
```

➤ Remove database

```
Remove-MailboxDatabase -Identity "OldDatabase"
```

3. Working with Mailbox

➤ Create mailbox

Enable-Mailbox –identity testuser1 –Database DB01 #Create a mailbox for an existing user

#Create a mailbox for an existing user

\$password = ConvertTo-SecureString "typethenewpasswordhere" -
AsPlainText -Force

New-Mailbox -UserPrincipalName jDoe@tala.local -Alias jDoe -Database
DB01 -Name "Doe, John" –organizationalunit " Utilisateurs " -Password
\$password -FirstName John -LastName Doe -DisplayName "Doe, John"

➤ Remove mailbox

Remove-Mailbox -Identity "John Doe"

➤ View mailbox

Get-Mailbox -Identity "John Doe"

➤ Move mailbox

```
New-moverrequest johndoe@test.be -targetdatabase DB01  
Get-mailbox | new moverrequest -targetdatabase DB01
```

```
Get-Mailbox -Database db02 -Arbitration | New-MoveRequest -  
TargetDatabase db01
```

```
Get-Mailbox -Database db02 -Monitoring | New-MoveRequest -  
TargetDatabase db01
```

```
Get-Mailbox -Database db02 -AuditLog | New-MoveRequest -  
TargetDatabase db01
```

➤ Get mailbox size

```
Get-Mailbox -ResultSize Unlimited | Get-MailboxStatistics |  
select DisplayName,TotalItemSize
```

Module 9 : VMware Powercli

Content

1. Introduction	p.143
2. Working with Host	p.144
3. Working with VM	p.146

1. Introduction

Vmware PowerCli is a PowerShell module that are available in the PowerGalery repository.

This module allows to query and manage vcenter and esxi object.

`Install-Module -Name VMware.PowerCLI`

2. Working with Host

- Connect to vcenter or ESXI

```
Connect-VIServer <vCenter_or_ESXi_FQDN/IP>
```

- Manage host

```
$esxi=Get-vmhost | fl * #View all information about the host
$esxi | Get-VMHostService #View all information about the
service of the host
$esxi | Get-VMHostService | where { $_.key -eq "TSM-SSH" } |
Set-VMHostService -Policy "On" # To autostart the service on
boot

$esxi | Get-VMHostService | where { $_.key -eq "TSM-SSH" } |
Start-VMHostService # To start the service now
```




➤ Manage networking

```
$esxi | Get-VirtualSwitch
$esxi | Get-VirtualPortGroup -VirtualSwitch vSwitch0
$esxi | New-VMHostNetworkAdapter -PortGroup "VMotion
Network" - VirtualSwitch vSwitch0 -IP 192.168.50.241
-SubnetMask 255.255.255.0 -VMotionEnabled $true #Add a
vmkernel portgroup and specify the role use by this vmkernel
$esxi | New-VirtualSwitch -Name vSwitch2 -Nic vmnic4,vmnic5
#Create virtualswitch
$esxi | Get-VirtualPortGroup -Name "VM Network" | Remove-
VirtualPortGroup -Confirm:$false #Remove portgroup
```

2. Working with VM

➤ Create a VM

```
New-VM -Name WinVM1 -NumCPU 2 -MemoryGB 4 -  
DiskGB 60 -NetworkName "Infrastructure Network -CD |  
Get-CDDrive | Set-CDDrive -IsoPath "[datastore0]/windows.iso"
```

➤ Manage VM

```
Get-VM WinVM1 | Start-VM  
Get-VM WinVM1 | Open-VMConsoleWindow
```

```
Get-VM WinVM1 | New-HardDisk -CapacityGB 20 –  
diskstorageformat thin –datastore Datastore0
```

➤ Changing guest ip address

```
$HostCredential = Get-Credential
$GuestCredential = Get-Credential
$ScriptText = 'Get-NetIPAddress |
Where-Object IPAddress -eq 192.168.2.37 |
Remove-NetIPAddress -Confirm:$false -PassThru |
New-NetIPAddress -IPAddress 10.10.2.5 '
-PrefixLength 22 '
-DefaultGateway 10.10.2.1'

Invoke-VMScript -HostCredential $hostCreds '
-GuestCredential $GuestCredential ' -ScriptText $ScriptText ' -VM VM001
```

Module 10 : COM and GUI in PowerShell

Content

- | | |
|---------------------------------|--------------|
| 1. PowerShell and GUI | p.150 |
| 2. Manipulate com object | p.156 |

1. PowerShell and GUI

- **Windows Presentation Foundation**

- User interface framework build inside .Net
- Defines all or most of the visible components in an Extensible Application Markup Language (XAML) document.
- Not loaded by default in PowerShell

`Add-Type -AssemblyName PresentationFramework`

- **About XAML**

- XML document that describe the components or elements of a user interface
- To display the UI you must
 - ✓ Create an xmldocument
 - ✓ Create a XMLNodeReader by casting the xml
 - ✓ Use the Xamlreader to create the user interface control from the document
 - ✓ Display the document as a windows

```
$xaml = [xml]'<?xml version="1.0" encoding="utf-8"?>
<Window
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
Width="350" Height="350">
<Label Content="Hello world" /></Window>'
$window = [System.Windows.Markup.XamlReader]::Load(
[System.Xml.XmlNodeReader]$xaml)
$window.ShowDialog()
```



- **Arrange information**

- **DockPanel**

- ✓ Used to arrange elements around the edge of a rectangle
 - ✓ Each element is positioned using the DockPanel.Dock attribute, and the possible values are Top, Bottom, Left, and Right.
 - ✓ By default, the last control in the DockPanel uses up the rest of the available space

- **Stackpanel**

- ✓ Allows one control to be placed next to another
 - ✓ Can be arranged horizontally or vertically.


```
'<?xml version="1.0" encoding="utf-8"?>
<Window
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
Width="800" Height="500">
<DockPanel>
<Label Height="50" Background="Gainsboro" DockPanel.Dock="Top" />
<StackPanel Background="LightBlue" Width="250" DockPanel.Dock="Left" />
<ComboBox Margin="5" DockPanel.Dock="Top" />
<StackPanel Height="30" Orientation="Horizontal" FlowDirection="RightToLeft"
DockPanel.Dock="Bottom">
    <Button Name="button1" Content="Exit" Width="50" Margin="5" />
    <Button Name="button2" Content="Cancel" Width="50" Margin="5" />
    <Button Name="button3" Content="OK" Width="50" Margin="5" />
</StackPanel>
<Label Background="LightCoral" DockPanel.Dock="Top" />
</DockPanel>
</Window>'
```



- **Handling events**

- An event is raised in a UI when a button is pressed
- To react to an event, an event handler must be created and attached to the control that raises the event

```
$xaml =[xml]<?xml version="1.0" encoding="utf-8"?>
<Window
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  Width="350" Height="350">
  <StackPanel>
    <Button Name="Button" Content="Run" />
    <Label Name="Label" />
  </StackPanel>
</Window>'

$reader=(New-Object System.Xml.XmlNodeReader $xaml)
$Window=[Windows.Markup.XamlReader]::Load( $reader )
$button = $Window.FindName("Button")
$button.add_Click({
    $window.findname('Label').content = 'Hello world'
})
$window.ShowDialog()
```

2. PowerShell and com object

- **Introduction**

- There are software components with .NET Framework and COM interfaces that enable you to perform many system administration tasks.
- Com object allow to connect to these objects and manipulate them in order to get or set properties without using cmdlets

Using library wscript.shell

```
$a = new-object -comobject wscript.shell
$a | get-member
AppActivate
CreateShortcut
Exec
LogEvent
Popup
RegDelete
RegRead
Run
Environment
CurrentDirectory
SpecialFolders

$a.popup("This is a test message from Dimitri ",0,"Test message",1)
```

Using library word

```
function Test-Spelling {  
    $wshell = New-Object -ComObject WScript.Shell  
    $word = New-Object -ComObject Word.Application  
    $word.Visible = $false  
    $doc = $word.Documents.Add()  
    $word.Selection.Paste()  
    if ($word.ActiveDocument.SpellingErrors.Count -gt 0)  
    {  
        $word.ActiveDocument.CheckSpelling()  
        $word.Visible = $false  
        $word.Selection.WholeStory()  
        $word.Selection.Copy()  
        $wshell.Popup( 'The spell check is complete, ' +  
            'the clipboard holds the corrected text.' ) }  
    else  
    { [void] $wshell.Popup('No Spelling Errors were detected.') }  
    $x = [ref] 0  
    $word.ActiveDocument.Close($x)  
    $word.Quit()  
}
```

References

- Don Jones, Jeffery D hicks (2013), *“Learn Windows PowerShell in a month of Lounches”*, Manning.
- Tyson Kopczynski, Pete Handley, Marco Shaw (2009), *“Windows PowerShell Unleashed”*, Sams.
- Ed Wilson (2008), *“Windows PowerShell Scripting Guide”*, Microsoft Press.
- Joezer Cookey-Gam, Brendan Keane, Jeffrey Rosen, Jonathan Runyon, Joel Stidley, (2008), *“Professional Windows PowerShell for Exchange Server 2007”*, Wrox.